دانشکده مهندسی کامپیوتر

عنوان

گزارش پروژه درس برنامه‌سازی موبایل

طراحی و پیاده‌سازی اپلیکیشن مدیریت و دنبال‌کردن فیلم و سریال «فیلم‌یاب»

اعضای گروه

مهرشاد دهقانی - ۴۰۱۱۰۵۹۱۲
ثنا بابایان ونستان - ۴۰۱۱۰۵۶۸۹
درسا شریفی قمبوانی - ۴۰۱۱۷۰۶۰۴

استاد

دکتر نجیمی

بهار ۱۴۰۵

معرفی پروژه

فیلم‌یاب یک اپلیکیشن فارسی و راست‌به‌چپ برای جست‌وجو و دنبال‌کردن فیلم‌ها و سریال‌ها است. کاربر مهمان می‌تواند محتوای عمومی را مشاهده و جست‌وجو کند و کاربر عضو، علاوه بر این امکانات، می‌تواند وضعیت تماشا، قسمت‌های دیده‌شده، امتیاز، نظر، علاقه‌مندی و فهرست‌های شخصی خود را مدیریت کند.

پروژه بر اساس مدل پیشرفته صورت‌مسئله طراحی و توسعه یافته است. در این مدل، اپلیکیشن موبایل مستقیماً با سرویس خارجی ارتباط ندارد؛ بلکه درخواست‌های آن به Backend اختصاصی فرستاده می‌شوند. Backend احراز هویت و ذخیره اطلاعات کاربران را انجام می‌دهد و برای اطلاعات عمومی آثار با OMDb API ارتباط برقرار می‌کند.

$$\text{Flutter Application} \longrightarrow \text{FastAPI Backend} \longrightarrow \text{OMDb API}$$
$$\text{FastAPI Backend} \longleftrightarrow \text{SQLite Database}$$

این جداسازی باعث می‌شود کلید OMDb در فایل اجرایی موبایل قرار نگیرد، اعتبارسنجی و کنترل دسترسی در سرور انجام شود و اطلاعات کاربران مستقل از یک دستگاه خاص در دیتابیس نگهداری شوند.

۱ دامنه امکانات

چهار بخش اصلی رابط کاربری شامل خانه، جست‌وجو، فهرست من و پروفایل است. صفحه خانه از کاتالوگ نمایشی داخل برنامه استفاده می‌کند. جست‌وجوی آنلاین و جزئیات آثار از Backend دریافت می‌شوند. در جست‌وجوی محلی عنوان، عنوان اصلی، ژانر، کارگردان و بازیگران بررسی می‌شوند، اما جست‌وجوی OMDb بر اساس عبارت عنوان انجام می‌شود.

صفحه جزئیات، مشخصات اصلی اثر را نمایش می‌دهد. برای سریال‌ها، درخواست جداگانه‌ای با گزینه دریافت قسمت‌ها ارسال می‌شود و Backend همه فصل‌های اعلام‌شده توسط OMDb را پیمایش می‌کند. اطلاعات هر قسمت شامل شماره فصل، شماره قسمت، عنوان، تاریخ انتشار و امتیاز IMDb است. پاسخ فصل‌های OMDb مدت زمان و خلاصه مستقل هر قسمت را ارائه نمی‌کند؛ در نتیجه Backend برای مدت زمان مقدار صفر و برای توضیح، تاریخ انتشار را به اپلیکیشن می‌فرستد.

۲ انواع کاربر

کاربر مهمان به مشاهده صفحه خانه، جست‌وجو و جزئیات دسترسی دارد، اما برای عملیات شخصی باید وارد حساب شود. کاربر عضو می‌تواند پروفایل و فعالیت‌های خود را مدیریت کند. نقش مدیر نیز در Backend تعریف شده و به مسیرهای مدیریت کاربران، گزارش‌ها، حذف نظر و آمار کلی دسترسی دارد. در نسخه فعلی برای امکانات مدیریتی از طریق هایAPI محافظت‌شده و مستندات Swagger در دسترس قرار می‌گیرند.

۳ اجزای اصلی سامانه

صورت‌مسئله مدل پیشرفته تنها به ساخت رابط موبایل محدود نیست و انتظار دارد یک Backend مستقل، دیتابیس سمت سرور، احراز هویت، نقش مدیر و واسطی برای سرویس اطلاعات فیلم وجود داشته باشد. معماری فیلم‌یاب این اجزا را در سه لایه موبایل، Backend و داده سازمان‌دهی می‌کند. جدول زیر مسئولیت اجزای اصلی سامانه را نشان می‌دهد.

جزء سامانه	نقش در پروژه
اپلیکیشن Flutter	نمایش رابط فارسی، دریافت ورودی، ناوبری، نگهداری state و ارتباط با API اختصاصی پروژه
Backend اختصاصی	ارائه API نسخه‌دار، اجرای منطق سرور، اعتبارسنجی، کنترل دسترسی و هماهنگی با OMDb

دیتابیس سمت سرور	نگهداری حساب‌ها، رسانه‌های کش‌شده، فعالیت‌ها، نظرها، فهرست‌ها و گزارش‌ها با SQLAlchemy
احراز هویت	ثبت‌نام و ورود، هش Argon۲، صدور JWT و تفکیک نقش کاربر و مدیر
کاتالوگ رسانه	جست‌وجو، دریافت جزئیات، نرمال‌سازی پاسخ OMDb و کش اطلاعات
سریال و قسمت	دریافت فصل‌ها بر اساس totalSeasons و ذخیره فصل و قسمت بدون داده ثابت
فعالیت کاربر	وضعیت تماشا، قسمت دیده‌شده، امتیاز، نظر، علاقه‌مندی و فهرست شخصی
مدیریت سامانه	مشاهده کاربران و گزارش‌ها، تغییر وضعیت حساب، حذف نظر و مشاهده آمار کلی از طریق API

۴ سناریوهای اصلی استفاده

در سناریوی استفاده عمومی، کاربر می‌تواند به عنوان مهمان وارد شود، آثار نمونه صفحه خانه را ببیند، در کاتالوگ جست‌وجو کند و وارد صفحه جزئیات شود. جست‌وجوی آنلاین از Backend عبور می‌کند و بنابراین حتی کاربر مهمان نیز کلید OMDb را در اختیار ندارد. این بخش به حساب و JWT وابسته نیست.
در سناریوی کاربر عضو، ثبت‌نام یا ورود ابتدا توکن دسترسی ایجاد می‌کند. پس از آن برنامه پروفایل، فهرست‌های شخصی و خلاصه فعالیت را دریافت می‌کند. کاربر می‌تواند یک فیلم را در وضعیت «بعداً می‌بینم» قرار دهد، برای آن امتیاز ثبت کند یا آن را به یک فهرست شخصی بیفزاید. برای سریال، جزئیات قسمت‌ها دریافت می‌شوند و هر قسمت دارای شناسه دیتابیسی است که در درخواست ثبت تماشا استفاده می‌شود.
در سناریوی مدیریتی، حساب دارای نقش admin می‌تواند از طریق Swagger یا یک کلاینت HTTP کاربران را ببیند، حساب را فعال یا غیرفعال کند، گزارش‌ها را بخواند، نظر گزارش‌شده را حذف کند و تعداد کاربران، رسانه‌های کش‌شده، نظرها و گزارش‌های باز را ببیند. این سناریو در سطح مدیریتی Backend و از طریق Swagger یا کلاینت مجاز API اجرا می‌شود.

معماری و ساختار کد

۵ اپلیکیشن Flutter

بخش موبایل با Flutter و Dart نوشته شده و از Material ۳، محلی‌سازی فارسی و جهت راست‌به‌چپ استفاده می‌کند. ساختار پوشه lib به صورت زیر است:

```
lib/
  data/        # presentation/demo catalogue
  models/      # MediaItem, Episode, Review, UserAccount
  screens/     # authentication and four main application areas
  services/    # backend and catalogue HTTP clients
  state/       # AppState and synchronization logic
  widgets/     # reusable media widgets
  main.dart
```

کلاس AppState از ChangeNotifier استفاده می‌کند و از طریق InheritedNotifier در دسترس صفحه‌ها قرار می‌گیرد. این کلاس نشست کاربر، رسانه‌های کش‌شده، وضعیت‌های تماشا، امتیازها، نظرات و فهرست‌ها را در حافظه نگه می‌دارد و تغییرات کاربر عضو را به Backend ارسال می‌کند.
دو سرویس شبکه در موبایل وجود دارد. OmdbService با وجود نام تاریخی خود، مستقیماً به OMDb متصل نمی‌شود و فقط مسیرهای /media در Backend پروژه را صدا می‌زند. BackendService نیز ثبت‌نام، ورود، پروفایل، فعالیت و فهرست‌های شخصی را مدیریت می‌کند.

۶ Backend

Backend با Python، FastAPI و SQLAlchemy نوشته شده است. تمام هاRouter در main.py با پیشوند مشترک /api/v1 ثبت می‌شوند؛ بنابراین پیشوندهای داخلی هر Router بخشی از مسیر نهایی را تعیین می‌کنند.

```
backend/
  app/
    routers/
      activity.py
      admin.py
      auth.py
      lists.py
      media.py
    services/omdb.py
    config.py
    database.py
    dependencies.py
    main.py
    models.py
    schemas.py
    security.py
  tests/test_api.py
  Dockerfile
  requirements.txt
```

در زمان شروع برنامه، جدول‌ها ساخته می‌شوند و اگر حساب مدیر تنظیم‌شده وجود نداشته باشد، حساب مدیر اولیه ایجاد می‌شود. FastAPI همچنین مستندات OpenAPI را در /docs و /redoc ارائه می‌کند. خطاهای HTTP و اعتبارسنجی با ساختار یکنواخت زیر برگردانده می‌شوند:

```
{
  "error": {
    "code": "validation_error",
    "message": "Request validation failed.",
    "details": []
  }
}
```

۷ تقسیم مسئولیت میان لایه‌ها

رابط Flutter مسئول نمایش داده، دریافت ورودی، ناوبری و نگهداری وضعیت موقت است. اعتبارسنجی ساده‌ای مانند خالی نبودن فیلد یا حداقل طول رمز در موبایل انجام می‌شود تا بازخورد سریع به کاربر داده شود. با این حال، این اعتبارسنجی جای کنترل سمت سرور را نمی‌گیرد؛ زیرا هر کلاینتی می‌تواند مستقیماً به API درخواست بفرستد. Backend همان قواعد را با مدل‌های Pydantic اعمال می‌کند و یکتا بودن ایمیل و نام کاربری را نیز با دیتابیس می‌سنجد.

Backend مسئول تشخیص هویت، مجوز، ماندگاری داده و هماهنگی با OMDb است. هاRouter فقط ورودی و خروجی عملیات را مدیریت می‌کنند و ارتباط مستقیم با OMDb در services/omdb.py متمرکز شده است. این جداسازی باعث می‌شود منطق سرویس خارجی در هاRouter تکرار نشود و تست آن با mock کردن متد _request امکان‌پذیر باشد.

SQLAlchemy لایه ماندگاری را فراهم می‌کند. Session دیتابیس به عنوان Dependency به هاendpoint تزریق می‌شود و بعد از پایان درخواست بسته می‌شود. این شیوه از ایجاد اتصال‌های پراکنده در هاRouter جلوگیری می‌کند و مسیر تغییر موتور دیتابیس را ساده‌تر نگه می‌دارد.

۸ جریان یک درخواست جست‌وجو

هنگام تایپ کاربر، SearchScreen تایمر قبلی را لغو می‌کند. پس از ۵۵۰ میلی‌ثانیه، AppState.search ابتدا نتایج محلی را محاسبه می‌کند و سپس OmdbService.search را فرا می‌خواند. این سرویس URL را بر اساس BACKEND_URL می‌سازد و درخواست زیر را می‌فرستد:

```
GET /api/v1/media/search?q=stranger
```

Router رسانه عبارت و شماره صفحه را اعتبارسنجی می‌کند. سرویس OMDb یک درخواست جست‌وجو می‌فرستد و برای هر نتیجه، اطلاعات کامل‌تر را با شناسه IMDb دریافت می‌کند. نتیجه نرمال‌شده در دیتابیس کش و به Flutter بازگردانده می‌شود. AppState شناسه‌های تکراری را حذف می‌کند، نتایج محلی و آنلاین را ترکیب می‌کند و های‌UI Listener را خبر می‌کند.
اگر Backend در دسترس نباشد، CatalogException به پیام فارسی تبدیل می‌شود و نتایج محلی همچنان نمایش داده می‌شوند. این رفتار به معنی آفلاین بودن کامل برنامه نیست؛ فقط اجازه می‌دهد محتوای نمونه و رسانه‌هایی که قبلاً در موبایل ذخیره شده‌اند از بین نروند.

۹ جریان یک عملیات احراز هویت‌شده

برای نمونه، ساخت فهرست با POST /api/v1/me/lists انجام می‌شود. BackendService توکن را در Header قرار می‌دهد. Dependency احراز هویت توکن را decode می‌کند، شناسه کاربر را از sub claim می‌خواند و سپس حساب فعال را از دیتابیس می‌گیرد. Router فهرست، نام را پاک‌سازی و رکورد CustomList را به همان کاربر متصل می‌کند. پس از commit، شناسه، نام و فهرست خالی شناسه رسانه‌ها به موبایل بازگردانده می‌شود.
این جریان نشان می‌دهد شناسه کاربر از body ارسالی موبایل گرفته نمی‌شود. در نتیجه کاربر نمی‌تواند با تغییر JSON، فهرستی برای کاربر دیگری بسازد. همین اصل در وضعیت تماشا، امتیاز، علاقه‌مندی و نظر نیز رعایت شده است.

۱۰ تنظیمات اجرا

تنظیمات از فایل env. یا متغیرهای محیطی خوانده می‌شوند. نام‌های واقعی تنظیمات عبارت‌اند از DATABASE_URL، JWT_SECRET، OMDB_API_KEY، OMDB_CACHE_HOURS، ACCESS_TOKEN_MINUTES، ADMIN_EMAIL، ADMIN_PASSWORD و ALLOWED_ORIGINS. مقدار پیش‌فرض اعتبار توکن ۴۳۲۰۰ دقیقه، معادل ۳۰ روز، است. آدرس پیش‌فرض Flutter برابر http://10.0.2.2:8000/api/v1 است. آدرس Android، در شبیه‌ساز 10.0.2.2 از داخل شبیه‌ساز به localhost رایانه میزبان نگاشت می‌شود؛ در نتیجه Uvicorn روی رایانه با http://127.0.0.1:8000 در دسترس است. برای دستگاه واقعی یا محیط انتشار می‌توان مقدار BACKEND_URL را با dart-define تغییر داد. **احراز هویت و همگام‌سازی**

۱۱ ثبت‌نام و ورود

در ثبت‌نام، نام، نام کاربری، ایمیل و رمز عبور به Backend ارسال می‌شوند. Pydantic طول و قالب ورودی را کنترل می‌کند و Backend تکراری نبودن ایمیل و نام کاربری را بررسی می‌کند. رمز عبور با الگوریتم پیشنهادی کتابخانه pwdlib، یعنی Argon۲، هش می‌شود و متن ساده آن ذخیره نمی‌شود.

ثبت‌نام و ورود موفق یک JWT برمی‌گردانند. توکن شامل شناسه کاربر، نقش و زمان انقضا است و Flutter آن را در Header درخواست‌های بعدی قرار می‌دهد:

```
Authorization: Bearer <access_token>
```

توکن و خلاصه‌ای از وضعیت برنامه در SharedPreferences ذخیره می‌شوند تا نشست پس از اجرای دوباره برنامه قابل بازیابی باشد. این ذخیره محلی نقش کش را دارد و منبع اصلی حساب کاربر نیست. اگر توکن ذخیره‌شده وجود داشته باشد، برنامه هنگام شروع پروفایل، فهرست‌ها و فعالیت کاربر را از Backend دریافت می‌کند. نشست قدیمی بدون JWT به عنوان حساب احراز هویت‌شده پذیرفته نمی‌شود.

۱۲ اعتبارسنجی حساب و پاسخ خطا

فرم Flutter نام حداقل دو کاراکتری، نام کاربری حداقل سه کاراکتری، ایمیل دارای علامت @ و رمز حداقل هشت کاراکتری را کنترل می‌کند. Backend کنترل دقیق‌تری دارد: ایمیل با EmailStr اعتبارسنجی می‌شود، طول رمز حداکثر ۱۲۸ کاراکتر است و نام کاربری فقط از حروف، اعداد، نویسه‌های فارسی، نقطه، خط تیره و زیرخط پذیرفته می‌شود. ایمیل پیش از ذخیره به حروف کوچک تبدیل می‌شود.
اگر ایمیل یا نام کاربری تکراری باشد، پاسخ با code status برابر ۴۰۹ و کد account_exists برمی‌گردد. ورود نادرست پاسخ ۴۰۱ با کد invalid_credentials دارد. خطای قالب ورودی نیز ۴۲۲ است و جزئیات فیلدهای نامعتبر در بخش details قرار می‌گیرند. BackendService کدهای رایج را به پیام فارسی تبدیل می‌کند تا متن خام انگلیسی سرور مستقیماً به کاربر نشان داده نشود.
رمز در هیچ مرحله‌ای در مدل UserAccount موبایل نگهداری نمی‌شود. فیلد تاریخی passwordHash در این مدل برای سازگاری داده‌های قدیمی وجود دارد، ولی حسابی که از پروفایل سرور ساخته می‌شود مقدار خالی برای آن دارد. منبع معتبر رمز فقط hash ثبت‌شده در جدول users است.

۱۳ ویرایش پروفایل

ویرایش نام، نام کاربری و زندگی‌نامه از طریق PATCH /api/v1/users/me انجام می‌شود. Backend یکتا بودن نام کاربری را بررسی می‌کند و پاسخ جدید پروفایل، وضعیت موبایل را به‌روز می‌کند. تغییر ایمیل و تصویر پروفایل در فرم فعلی Flutter ارائه نشده است، هرچند مدل Backend فیلد URL تصویر را دارد.
ویرایش پروفایل پس از بسته شدن Sheet Bottom انجام می‌شود. این ترتیب برای جلوگیری از تغییر Inherited-Notifier در حالی انتخاب شده است که هایWidget داخل overlay هنوز به آن وابسته‌اند. این همان مسئله‌ای بود که می‌توانست assertion مربوط به _dependents.isEmpty را در Flutter ایجاد کند. اکنون Bottom Sheet فقط یک شیء draft برمی‌گرداند و صفحه والد درخواست Backend و تغییر state را انجام می‌دهد.

۱۴ الگوی همگام‌سازی فعالیت

ایجاد و حذف فهرست و تغییر عضویت یک اثر در فهرست منتظر پاسخ Backend می‌مانند و در صورت خطا پیام مناسب نمایش داده می‌شود. در مقابل، وضعیت تماشا، علاقه‌مندی، امتیاز، نظر و قسمت دیده‌شده ابتدا به صورت خوش‌بینانه در وضعیت محلی تغییر می‌کنند و سپس درخواست سرور در پس‌زمینه ارسال می‌شود. نتیجه درخواست شبکه از طریق syncError قابل ثبت و نمایش است تا وضعیت ارتباط با سرور برای لایه رابط کاربری مشخص باشد.
هنگام شروع نشست، مسیر me/activity/ وضعیت‌های تماشا، شناسه قسمت‌های دیده‌شده، امتیازها و علاقه‌مندی‌ها را برمی‌گرداند. نگاشت شناسه قسمت‌های دیتابیس به شناسه نمایشی موبایل زمانی ممکن است که جزئیات همان سریال قبلاً در کش موبایل وجود داشته باشد.

۱۵ مدیریت نشست و خروج

توکن نشست همراه تاریخ انقضای محلی نگهداری می‌شود و در شروع برنامه برای بازیابی حساب استفاده می‌گردد. هنگام خروج، حساب فعال، توکن و تاریخ نشست از state پاک می‌شوند و BackendService نیز دیگر Header احراز هویت ارسال نمی‌کند. داده عمومی کش‌شده می‌تواند برای نمایش کاتالوگ باقی بماند، ولی دسترسی به عملیات شخصی تا ورود بعدی مسدود می‌شود.

رسانه، فصل‌ها و کش

۱۶ جست‌وجو و جزئیات

صفحه جست‌وجو پس از ۵۵۰ میلی‌ثانیه توقف تایپ، عبارت را به Backend می‌فرستد. نتایج محلی و آنلاین با شناسه IMDb ادغام می‌شوند و کاربر می‌تواند آن‌ها را بر اساس فیلم یا سریال فیلتر کند. در صورت خطای شبکه، نتایج محلی نمایش داده می‌شوند.

Backend برای جست‌وجو ابتدا مسیر جست‌وجوی OMDb را صدا می‌زند و سپس برای هر نتیجه، جزئیات آن را دریافت و نرمال‌سازی می‌کند. ساختار نهایی شامل شناسه، عنوان، نوع، پوستر، خلاصه، سال، ژانر، امتیاز، زمان، کشور، کارگردان، بازیگران و تعداد فصل‌های اعلام‌شده است. backdropUrl در پاسخ فعلی خالی است، زیرا OMDb تصویر پس‌زمینه جداگانه‌ای ارائه نمی‌کند.

پاسخ جست‌وجوی OMDb به تنهایی اطلاعاتی مانند کارگردان، بازیگران، خلاصه کامل و زمان اثر را ندارد. به همین دلیل Backend پس از دریافت فهرست اولیه، برای هر شناسه مسیر جزئیات را اجرا می‌کند. این تصمیم پاسخ غنی و یکسانی به موبایل می‌دهد، اما تعداد درخواست‌های خارجی را افزایش می‌دهد و باید در محاسبه quota سرویس OMDb در نظر گرفته شود. کش Backend بخشی از این هزینه را در جست‌وجوهای تکراری کاهش می‌دهد.

پارامتر page در Backend از ۱ تا ۱۰۰ پذیرفته می‌شود و مقدار totalResults سرویس خارجی به عدد تبدیل می‌شود. پارامتر صفحه در API امکان توسعه نمایش صفحه‌های بعدی نتایج را فراهم می‌کند و صفحه جست‌وجوی موبایل درخواست اولیه را با مقدار پیش‌فرض صفحه ارسال می‌کند.

۱۷ نرمال‌سازی پاسخ OMDb

سرویس خارجی برای داده ناموجود معمولاً رشته N/A برمی‌گرداند. لایه نرمال‌سازی این مقادیر را به رشته خالی، لیست خالی یا مقدار عددی صفر تبدیل می‌کند تا مدل Dart مجبور نباشد در تمام صفحه‌ها حالت N/A را جداگانه کنترل کند. سال با استخراج عدد چهاررقمی خوانده می‌شود و در سریال دارای بازه، اولین سال به year و آخرین سال به endYear می‌رود.

فیلد داخلی	منبع OMDb	تبدیل
id	imdbID	شناسه اصلی اثر
originalTitle / title	Title	در نسخه فعلی هر دو از یک مقدار ساخته می‌شوند
type	Type	مقدار series به سریال و سایر مقادیر به movie تبدیل می‌شوند
posterUrl	Poster	مقدار N/A به رشته خالی تبدیل می‌شود
overview	Plot	خلاصه full در درخواست جزئیات
cast / genres	Actors / Genre	رشته جداشده با ویرگول به List تبدیل می‌شود
rating	imdbRating	تبدیل امن رشته به double
runtime	Runtime	استخراج بخش عددی زمان

declaredSeasonCount	totalSeasons	تعداد فصل اعلام‌شده برای پیمایش فصل‌ها

برای آثار آنلاین، عنوان استاندارد OMDb در مدل قرار می‌گیرد. ترجمه‌های فارسی موجود در صفحه خانه مربوط به کاتالوگ نمایشی داخل پروژه‌اند؛ بنابراین رابط می‌تواند در کنار داده واقعی آنلاین، محتوای فارسی آماده نیز ارائه کند.

۱۸ دریافت تمام قسمت‌ها

برای جزئیات سریال، Flutter پارامتر include_episodes=true را ارسال می‌کند. Backend مقدار totalSeasons را از پاسخ اصلی می‌خواند و فصل‌های ۱ تا آن مقدار را بدون فهرست ثابت یا داده hard-code شده درخواست می‌کند. فصل و قسمت‌ها در جدول‌های جداگانه ذخیره می‌شوند و شناسه داخلی قسمت نیز به موبایل بازگردانده می‌شود تا ثبت قسمت دیده‌شده ممکن باشد.
فصل صفر در این منطق ساخته نمی‌شود. خطاهای HTTP یا پاسخ نامعتبر OMDb نیز به پاسخ استاندارد سرویس خارجی تبدیل می‌شوند تا Flutter بتواند پیام خطا و گزینه تلاش مجدد را نمایش دهد.
قبل از ذخیره مجدد فصل‌ها، Episode و هایSeason قبلی همان رسانه حذف می‌شوند تا داده قدیمی و جدید مخلوط نشوند. ابتدا رکورد Media ایجاد یا به‌روزرسانی و flush می‌شود تا Key Foreign فصل و قسمت به رکورد موجود اشاره کند. برای هر فصل یک رکورد Season و برای هر عضو آرایه Episodes یک رکورد Episode ساخته می‌شود. محدودیت یکتا روی ترکیب رسانه، شماره فصل و شماره قسمت از تکرار منطقی جلوگیری می‌کند.
شناسه دیتابیسی Episode در پاسخ databaseId قرار می‌گیرد. مدل Dart علاوه بر شناسه نمایشی season-number، این شناسه را نگه می‌دارد. هنگام علامت‌گذاری قسمت، فقط اگر databaseId موجود باشد درخواست Backend فرستاده می‌شود. به همین دلیل قسمت‌های واقعی دریافت‌شده از سرور قابل ثبت هستند، ولی یک Episode صرفاً محلی و بدون شناسه سرور نمی‌تواند به جدول watched_episodes متصل شود.

۱۹ کش Backend و موبایل

جدول media چند ستون شاخص شامل شناسه ،IMDb نوع و عنوان دارد و پاسخ نرمال‌شده کامل را به صورت JSON در ستون payload ذخیره می‌کند. زمان کش و زمان انقضا نیز نگهداری می‌شوند. پیش‌فرض اعتبار کش ۲۴ ساعت است.
اگر رکورد هنوز منقضی نشده باشد، Backend از آن استفاده می‌کند. برای یک سریال که قسمت‌هایش درخواست شده‌اند، قسمت‌ها از جدول episodes خوانده می‌شوند. پس از انقضا، Backend داده را از OMDb تازه‌سازی می‌کند و زمان اعتبار جدیدی برای رکورد در نظر می‌گیرد.
در خواندن کش سریال، تنها وجود payload اصلی کافی نیست. اگر include_episodes فعال باشد و هیچ Episode متناظری در دیتابیس وجود نداشته باشد، سرویس miss cache در نظر می‌گیرد و داده را دوباره از OMDb می‌گیرد. این رفتار از نمایش سریال بدون قسمت در حالی جلوگیری می‌کند که کاربر صراحتاً قسمت‌ها را درخواست کرده است.
Flutter نیز رسانه‌های دریافت‌شده را در SharedPreferences ذخیره می‌کند تا در فهرست‌ها و هنگام خطای جست‌وجو قابل نمایش باشند. تصاویر با سازوکار استاندارد شبکه Flutter بارگذاری می‌شوند و placeholder و حالت خطا دارند؛ کتابخانه جداگانه‌ای برای کش پایدار تصویر اضافه نشده است.

مدل داده و فعالیت کاربر

۲۰ مدل‌های دیتابیس

جدول users اطلاعات حساب، هش رمز، نقش، وضعیت فعال بودن و تاریخ ساخت را نگهداری می‌کند. جدول‌های media، seasons و episodes کش اطلاعات عمومی آثار هستند. برخلاف یک مدل کاملاً نرمال‌شده، تمام فیلدهای

رسانه مانند ژانر و بازیگران ستون مستقل ندارند و عمدتاً داخل media.payload قرار می‌گیرند.
فعالیت شخصی در جدول‌های مستقل ذخیره می‌شود. محدودیت یکتا روی ترکیب کاربر و رسانه یا کاربر و قسمت، از ثبت چند رکورد متناقض جلوگیری می‌کند.

مدل	کاربرد واقعی
User	حساب، پروفایل، نقش و وضعیت فعال بودن
Media	مشخصات شاخص و payload کش‌شده OMDb
Episode / Season	شماره فصل و اطلاعات پایه قسمت‌ها
WatchStatus	وضعیت یک رسانه برای یک کاربر
WatchedEpisode	دیده‌شدن یک قسمت دیتابیسی برای یک کاربر
Review / Rating	امتیاز یک تا پنج و نظر دارای پرچم اسپویل
Favorite	رابطه علاقه‌مندی مستقل از جدول وضعیت
CustomList / CustomListItem	فهرست شخصی و شناسه رسانه‌های آن
Report	گزارش یک نظر توسط کاربر

کلید خارجی مستقیم از WatchStatus، Rating، Review، Favorite و ListItem به جدول Media تعریف نشده و شناسه IMDb به صورت رشته نگهداری می‌شود. در نتیجه نمودار رابطه این فعالیت‌ها با Media یک رابطه منطقی از طریق شناسه است، نه الزاماً Key Foreign دیتابیس.

۲۱ جزئیات جدول کاربران و رسانه

جدول کاربران علاوه بر نام، نام کاربری و ایمیل، فیلدهای bio، avatar_url، role، is_active و created_at را دارد. ایمیل و نام کاربری unique و index شده‌اند. غیرفعال کردن کاربر رکورد او را حذف نمی‌کند؛ Dependency احراز هویت پس از خواندن حساب، مقدار is_active را بررسی می‌کند و برای حساب غیرفعال پاسخ ۴۰۱ می‌دهد.
جدول Media از شناسه IMDb به عنوان key primary استفاده می‌کند. ستون‌های media_type و title برای شناسایی سریع و ستون payload برای ذخیره JSON کامل هستند. cached_at زمان ساخت رکورد و expires_at زمان پایان اعتبار را نگه می‌دارد. در به‌روزرسانی فعلی مقدار expires_at تغییر می‌کند، اما cached_at به صورت خودکار به زمان refresh جدید تغییر داده نمی‌شود؛ بنابراین معیار واقعی تازگی در منطق سرویس همان expires_at است.
جدول Season شناسه داخلی، شناسه رسانه و شماره فصل را دارد. Episode نیز شناسه داخلی، شناسه IMDb اختیاری، شناسه رسانه، شماره فصل، شماره قسمت، عنوان، تاریخ انتشار و امتیاز IMDb را ذخیره می‌کند. Episode مستقیماً Key Foreign به Season ندارد و ارتباط فصل آن از ترکیب media_id و season_number برداشت می‌شود.

۲۲ یکتایی و مالکیت داده‌های کاربر

برای WatchStatus، Rating و Favorite روی ترکیب user_id و media_id محدودیت یکتا وجود دارد. در نتیجه درخواست تکراری رکورد جدید نمی‌سازد؛ endpoint رکورد قبلی را به‌روزرسانی می‌کند یا همان وضعیت را حفظ می‌کند. WatchedEpisode نیز روی کاربر و Episode یکتا است و مقدار Boolean آن تغییر می‌کند.
Review برخلاف Rating امکان ثبت بیش از یک نظر برای هر کاربر و رسانه را فراهم می‌کند؛ بنابراین یک کاربر می‌تواند چند نظر برای یک اثر ثبت کند. Report به یک Review مشخص و گزارش‌کننده متصل است. وضعیت گزارش با مقدار اولیه open ذخیره می‌شود و در فهرست گزارش‌های مدیریتی قابل مشاهده است.

CustomList به کاربر مالک متصل است. قبل از خواندن، حذف یا تغییر آیتم، تابع owned_list بررسی می‌کند که شناسه فهرست واقعاً متعلق به کاربر جاری باشد. ListItem روی ترکیب شناسه فهرست و شناسه رسانه unique است، پس افزودن دوباره یک فیلم به همان فهرست، آیتم تکراری تولید نمی‌کند.

۲۳ وضعیت تماشا و علاقه‌مندی

مقادیر وضعیت تماشا عبارت‌اند از none، plan، watching، completed، paused، dropped و favorite. در مدل موبایل، علاقه‌مندی یکی از وضعیت‌های انحصاری است؛ هم‌زمان Backend برای سازگاری با API یک رکورد در جدول favorites نیز نگهداری می‌کند. بنابراین انتخاب علاقه‌مندی جای وضعیت قبلی را در رابط فعلی می‌گیرد و یک برچسب مستقل در کنار وضعیت تماشا نیست.
پیشرفت سریال از نسبت تعداد قسمت‌های علامت‌خورده به کل قسمت‌های دریافت‌شده محاسبه می‌شود:

$$\text{Progress} = \frac{\text{Episodes Watched}}{\text{Episodes Loaded}}$$

برای فیلم بدون قسمت، وضعیت completed به معنی پیشرفت کامل است.
در پروفایل، تعداد فیلم دیده‌شده با شمردن فیلم‌هایی محاسبه می‌شود که وضعیت completed دارند. تعداد سریال دنبال‌شده بر اساس پیشرفت بزرگ‌تر از صفر است. زمان تقریبی نیز از runtime فیلم‌های کامل‌شده و قسمت‌های علامت‌خورده جمع می‌شود. چون قسمت‌های OMDb در پاسخ فعلی runtime صفر دارند، زمان تماشای سریال‌های آنلاین ممکن است کمتر از مقدار واقعی محاسبه شود. ژانر محبوب از فراوانی ژانر رسانه‌هایی به دست می‌آید که وضعیتشان none نیست و میانگین امتیاز از Map محلی ratings محاسبه می‌شود.

۲۴ نظر، امتیاز و فهرست

Backend امتیاز را در بازه یک تا پنج اعتبارسنجی می‌کند. نظر شامل متن و پرچم اسپویل است. هنگام ثبت نظر، موبایل آن را فوراً در فهرست محلی صفحه قرار می‌دهد و هم‌زمان درخواست سرور را ارسال می‌کند. مسیر عمومی دریافت نظرات در Backend وجود دارد و ثبت نظر از صفحه جزئیات به همین مسیر ارسال می‌شود. نظر ثبت‌شده در state و SharedPreferences موبایل نیز قرار می‌گیرد تا در همان رابط و اجراهای بعدی دستگاه نمایش داده شود.
فهرست‌های شخصی از مسیرهای Backend ایجاد، خوانده و حذف می‌شوند. تغییر عضویت از Sheet Bottom صفحه جزئیات نیز با شناسه IMDb روی سرور انجام می‌شود. هنگام ورود یا بازیابی نشست، نام فهرست‌ها، شناسه دیتابیسی آن‌ها و شناسه آثار دوباره از سرور خوانده می‌شوند. اگر اطلاعات یک اثر هنوز در کاتالوگ نمونه یا کش موبایل نباشد، نام آن در فهرست دیده نمی‌شود تا زمانی که جزئیات اثر دریافت و کش شود.
صفحه Library پنج دیدگاه دارد. چهار دیدگاه اول از Map وضعیت‌ها فیلتر می‌شوند و دیدگاه پنجم هاCustomList را نشان می‌دهد. این طراحی ساده است، ولی چون favorite در همان enum وضعیت قرار دارد، یک اثر در هر لحظه فقط در یکی از تب‌های وضعیت یا علاقه‌مندی قرار می‌گیرد. جدول Favorite در Backend امکان نگهداری رابطه علاقه‌مندی را به صورت مستقل در لایه سرور فراهم می‌کند.

API و کنترل دسترسی

جدول زیر مسیرهای واقعی نسخه فعلی را خلاصه می‌کند. تمام مسیرهای بخش کاربر و مدیر به JWT نیاز دارند؛ جست‌وجو، جزئیات رسانه و خواندن نظرات عمومی هستند.

Method	Path	کاربرد
POST	/api/v۱/auth/register	ثبت‌نام و دریافت توکن
POST	/api/v۱/auth/login	ورود و دریافت توکن
GET / PATCH	/api/v۱/users/me	دریافت یا ویرایش پروفایل

Method	Path	کاربرد
GET	/api/v۱/media/search?q=...	جست‌وجوی رسانه
GET	/api/v۱/media/{imdb_id}	جزئیات و قسمت‌های اختیاری
PUT	/api/v۱/me/watch-status/{media_id}	ثبت وضعیت تماشا
PUT	/api/v۱/me/episodes/{episode_id}/watched	ثبت وضعیت قسمت
PUT	/api/v۱/me/ratings/{media_id}	ثبت امتیاز
PUT / DELETE	/api/v۱/me/favorites/{media_id}	تغییر علاقه‌مندی
POST / GET	/api/v۱/media/{media_id}/reviews	ثبت یا خواندن نظر
GET	/api/v۱/me/activity	همگام‌سازی فعالیت
GET / POST	/api/v۱/me/lists	خواندن یا ساخت فهرست
DELETE	/api/v۱/me/lists/{list_id}	حذف فهرست
PUT	/api/v۱/me/lists/{list_id}/items	افزودن اثر به فهرست
DELETE	/api/v۱/me/lists/{list_id}/items/{media_id}	حذف اثر از فهرست
POST	/api/v۱/reports	گزارش یک نظر
GET	/api/v۱/admin/users	فهرست کاربران
PATCH	/api/v۱/admin/users/{user_id}/active	تغییر فعال بودن حساب
GET	/api/v۱/admin/reports	مشاهده گزارش‌ها
DELETE	/api/v۱/admin/reviews/{review_id}	حذف نظر
GET	/api/v۱/admin/stats	آمار کلی

وابستگی CurrentUser توکن Bearer را اعتبارسنجی می‌کند، شناسه کاربر را از claim با نام sub می‌خواند و فعال بودن حساب را بررسی می‌کند. وابستگی AdminUser علاوه بر آن، مقدار نقش را با admin مقایسه می‌کند. قرار دادن نقش داخل JWT به تنهایی مبنای مجوز نیست، زیرا کاربر مجدداً از دیتابیس خوانده می‌شود و نقش فعلی او بررسی می‌شود.
گزارش نظر و مسیرهای مدیریتی در Backend در گروه جداگانه‌ای قرار دارند و از طریق احراز هویت نقش مدیر و مستندات تعاملی Swagger استفاده می‌شوند.

۲۵ معنای عملیات HTTP

در مسیرهای فعالیت از PUT برای عملیات idempotent استفاده شده است. ثبت دوباره همان امتیاز یا وضعیت، نتیجه منطقی یکسانی دارد و رکورد تکراری ایجاد نمی‌کند. برای حذف علاقه‌مندی و آیتم فهرست از DELETE استفاده می‌شود و پاسخ موفق بدون محتوا code status برابر ۲۰۴ دارد. ایجاد حساب، نظر، گزارش و فهرست با POST انجام می‌شود، زیرا هر بار یک منبع جدید ایجاد می‌کنند.
مسیر PUT /me/watch-status/{media_id} مقدار none را به صورت حذف رکورد تفسیر می‌کند. سایر مقادیر رکورد موجود را update می‌کنند. مسیر قسمت دیده‌شده برخلاف آن رکورد را هنگام false حذف نمی‌کند، بلکه فیلد watched را false قرار می‌دهد. در خروجی /me/activity فقط رکوردهای true به عنوان قسمت دیده‌شده برگردانده می‌شوند.
در هایendpoint Rating و ،WatchStatus شناسه رسانه با regex مدل ورودی اعتبارسنجی نشده است، در حالی که ListItem الزام tt[0-9]+ دارد و سرویس جزئیات نیز همین قالب را بررسی می‌کند. شناسه‌هایی که UI استفاده می‌کند از داده IMDb دریافت می‌شوند و در نتیجه مسیرهای فعالیت نیز شناسه استاندارد همان رسانه را ذخیره می‌کنند.

۲۶ گزارش و مدیریت

برای ثبت گزارش، کاربر شناسه نظر و دلیل حداقل سه کاراکتری را می‌فرستد. Backend ابتدا وجود Review را بررسی می‌کند و سپس Report را با شناسه گزارش‌کننده و وضعیت open ذخیره می‌کند. پاسخ تنها شناسه گزارش و وضعیت آن را برمی‌گرداند.
فهرست کاربران مدیر pagination ساده دارد و حداکثر صد کاربر در هر درخواست برمی‌گرداند. تغییر وضعیت حساب با parameter query به نام active انجام می‌شود. آمار مدیر شامل تعداد کل کاربران، تعداد رسانه‌های کش‌شده، تعداد نظرها و تعداد گزارش‌های باز است. حذف نظر وجود آن را بررسی می‌کند و پس از حذف پاسخ ۲۰۴ می‌دهد.
ایجاد خودکار مدیر در lifespan برنامه بر اساس ایمیل تنظیم‌شده انجام می‌شود. اگر کاربری با همان ایمیل وجود داشته باشد، برنامه مدیر دیگری نمی‌سازد و نقش آن کاربر را نیز به صورت خودکار تغییر نمی‌دهد. بنابراین متغیرهای مدیر باید پیش از اولین اجرای دیتابیس با دقت تنظیم شوند.

رابط کاربری و تجربه کاربر

برنامه به طور کامل در حالت تیره اجرا می‌شود و locale فارسی، هایdelegate محلی‌سازی و Directionality راست‌به‌چپ در سطح برنامه تنظیم شده‌اند. فیلدهای نام، نام کاربری، زندگی‌نامه، نظر و نام فهرست از ورودی فارسی پشتیبانی می‌کنند؛ ایمیل و رمز عبور به صورت چپ‌به‌راست نمایش داده می‌شوند.
صفحه ورود بین حالت ثبت‌نام و ورود جابه‌جا می‌شود و هنگام درخواست شبکه دکمه را غیرفعال و نشانگر پیشرفت نمایش می‌دهد. حالت مهمان بدون ساخت حساب وارد پوسته اصلی می‌شود. عملیات عضو برای مهمان مسدود است و او به ورود یا ثبت‌نام هدایت می‌شود.
صفحه اصلی دارای بخش شاخص، ادامه تماشا و مجموعه‌های پوستر است. صفحه جست‌وجو وضعیت بارگذاری، فیلتر نوع رسانه، خطای شبکه و نتیجه خالی را مدیریت می‌کند. صفحه جزئیات برای بارگذاری قسمت‌ها نشانگر پیشرفت، پیام خطا و دکمه تلاش مجدد دارد. صفحه فهرست دارای تب‌های در حال تماشا، بعداً می‌بینم، دیده‌شده، علاقه‌مندی و شخصی است. صفحه پروفایل نیز آمار را از وضعیت فعلی موبایل محاسبه می‌کند، نه از endpoint آمار مدیر.
برای جلوگیری از خطای lifecycle در Dialog و Sheet، Bottom داده فرم ابتدا با Navigator.pop به صفحه والد برگردانده می‌شود و سپس وضعیت کلی تغییر می‌کند. هاGrid و کارت‌های آماری نیز با نسبت ابعاد مناسب طراحی شده‌اند تا در عرض‌های کوچک overflow ایجاد نشود.

۲۷ صفحه ورود و حالت مهمان

RootRouter بر اساس دو مقدار signedIn و guest تصمیم می‌گیرد که AuthScreen یا MainShell نمایش داده شود. فرم ورود و ثبت‌نام در یک صفحه مشترک قرار دارند. در حالت ثبت‌نام دو فیلد نام و نام کاربری نیز دیده می‌شوند. هنگام ارسال، اگر Form معتبر باشد، متغیر submitting فعال می‌شود، دکمه غیرفعال و یک CircularProgressIndicator کوچک نمایش داده می‌شود. این رفتار از ارسال چندباره فرم جلوگیری می‌کند.
در حالت مهمان هیچ JWT ساخته نمی‌شود. AppState مقدار guest را محلی ذخیره می‌کند تا پوسته اصلی قابل نمایش باشد، اما account برابر null باقی می‌ماند. تابع requireMember و کنترل‌های صفحه جزئیات اجازه تغییر فعالیت را نمی‌دهند. در صفحه پروفایل مهمان، دکمه ورود یا ثبت‌نام با خروج از حالت مهمان کاربر را به AuthScreen برمی‌گرداند.

۲۸ صفحه خانه و کارت رسانه

HomeScreen از کاتالوگ نمونه و وضعیت فعلی AppState استفاده می‌کند. محتوای شاخص از آثار دارای featured=true انتخاب می‌شود. ادامه تماشا بر اساس وضعیت watching یا پیشرفت قسمت‌ها شکل می‌گیرد و مجموعه‌های فیلم، سریال و آثار جدید در ردیف یا Grid نمایش داده می‌شوند.

MediaPosterCard و PosterImage اجزای مشترک هستند. PosterImage در حین بارگذاری placeholder و در خطا آیکن مناسب نشان می‌دهد. کارت پوستر از Navigator برای باز کردن DetailScreen استفاده می‌کند؛ بنابراین صفحات خانه، جست‌وجو و فهرست از یک نمایش یکسان رسانه استفاده می‌کنند.
کاتالوگ نمونه ترجمه فارسی، تصویر TMDb و اطلاعات نمایشی آماده دارد، اما داده آنلاین از OMDb می‌آید. این دو منبع در AppState با شناسه IMDb ادغام می‌شوند. اگر یک شناسه هم در کاتالوگ نمونه و هم در savedMedia باشد، نسخه نمونه در allMedia مقدم است. در صفحه جزئیات سریال، نسخه کامل دریافت‌شده به متغیر محلی صفحه و سپس کش savedMedia داده می‌شود.

۲۹ صفحه جست‌وجو

TextField صفحه جست‌وجو ورودی فارسی را راست‌چین دریافت می‌کند. هر تغییر باعث بازسازی فوری UI برای نمایش دکمه پاک کردن می‌شود، ولی درخواست شبکه تا پایان debounce اجرا نمی‌شود. سه ChoiceChip برای همه، فیلم و سریال وجود دارند و فیلتر روی نتیجه ترکیبی اعمال می‌شود.
وقتی عبارت خالی است، کاتالوگ نمونه نمایش داده می‌شود. برای عبارت غیرخالی، AppState ابتدا میان allMedia جست‌وجوی محلی انجام می‌دهد و بعد نتیجه Backend را اضافه می‌کند. LinearProgressIndicator وضعیت جست‌وجوی شبکه را نشان می‌دهد. در SocketException، TimeoutException یا پاسخ نامعتبر، پیام فارسی نمایش داده می‌شود و نتیجه محلی حفظ می‌شود.
پیکربندی سرویس بر اساس BACKEND_URL انجام می‌شود. بنابراین تمام درخواست‌های آنلاین صفحه جست‌وجو به API اختصاصی پروژه فرستاده می‌شوند و هیچ کلید OMDb در اپلیکیشن موبایل قرار نمی‌گیرد.

۳۰ صفحه جزئیات و قسمت‌ها

DetailScreen یک StatefulWidget است، زیرا رسانه اولیه ممکن است فقط خلاصه اطلاعات سریال را داشته باشد. در didChangeDependencies، تنها یک بار بارگذاری قسمت‌ها آغاز می‌شود. سه حالت loading، error و success نگهداری می‌شوند. گزینه تلاش مجدد پرچم درخواست را پاک و همان عملیات را دوباره اجرا می‌کند.
کاربر عضو از این صفحه می‌تواند علاقه‌مندی را تغییر دهد، وضعیت تماشا را از RadioGroup انتخاب کند، امتیاز یک تا پنج بدهد، نظر دارای اسپویل ثبت کند، قسمت را علامت بزند و عضویت اثر در چند فهرست شخصی را تغییر دهد. Dialog نظر و Sheet Bottom فهرست قبل از تغییر AppState بسته می‌شوند تا وابستگی overlay به InheritedNotifier هنگام notifyListeners باقی نماند.
نظر اسپویل در کارت نظر ابتدا پنهان است و کاربر با اقدام صریح آن را آشکار می‌کند. این پنهان‌سازی فقط در UI است و متن کامل نظر در پاسخ Backend وجود دارد؛ بنابراین پرچم spoiler یک سیاست نمایش است، نه رمزگذاری محتوا.

۳۱ صفحه فهرست و پروفایل

LibraryScreen با TabController پنج‌تب ساخته شده است. هر StatusGrid از allMedia رسانه‌های دارای وضعیت مربوط را جدا می‌کند. فهرست‌های شخصی به صورت ExpansionTile نمایش داده می‌شوند و ساخت فهرست با Dialog فارسی انجام می‌شود. ایجاد و حذف فهرست و مدیریت عضویت آثار از طریق شناسه فهرست و شناسه IMDb با Backend هماهنگ می‌شوند.
ProfileScreen نام، نام کاربری، زندگی‌نامه و ایمیل را نمایش می‌دهد. آمار شامل فیلم کامل‌شده، سریال دنبال‌شده، قسمت دیده‌شده، زمان تقریبی، ژانر محبوب و میانگین امتیاز است. این اعداد از state موبایل محاسبه می‌شوند؛ endpoint فعالیت سرور بخشی از داده را فراهم می‌کند، اما پروفایل درخواست مستقلی برای آمار تجمیعی ندارد. خروج امن، توکن و نشست محلی را پاک می‌کند ولی داده حساب را از Backend حذف نمی‌کند.

آزمون و ارزیابی

۳۲ آزمون‌های Flutter

فایل widget_test.dart در حال حاضر ۹ آزمون دارد. این آزمون‌ها صفحه احراز هویت فارسی، اعتبارسنجی ثبت‌نام، حفظ حساب هنگام استفاده مهمان، نبود قسمت hard-code شده در کاتالوگ نمونه، ثبت نظر فارسی، ساخت فهرست و افزودن اثر، بازیابی سریال آنلاین از کش، نبود overflow در صفحه باریک و ویرایش امن پروفایل را بررسی می‌کنند.
FakeBackend عملیات حساب، پروفایل، فعالیت و فهرست را در حافظه شبیه‌سازی می‌کند تا آزمون‌های Flutter به سرور واقعی وابسته نباشند. این fake جست‌وجو و جزئیات OMDb را شبیه‌سازی نمی‌کند؛ این دو عملیات توسط سرویس کاتالوگ جداگانه انجام می‌شوند و در آزمون‌های فعلی درخواست آنلاین ندارند.

۳۳ آزمون‌های Backend

فایل tests/test_api.py دو سناریوی یکپارچه دارد. سناریوی اول ثبت‌نام، خواندن و ویرایش پروفایل، جلوگیری از حساب تکراری، ساخت فهرست، افزودن اثر، اعتبارسنجی امتیاز و رد دسترسی مدیر برای کاربر عادی را بررسی می‌کند. سناریوی دوم پاسخ OMDb را mock می‌کند و نرمال‌سازی سریال، دریافت همه فصل‌های اعلام‌شده و استفاده مجدد از کش را می‌سنجد.
ترکیب این دو سناریو مسیرهای اصلی احراز هویت، پروفایل، فهرست، اعتبارسنجی، کنترل نقش و سرویس رسانه را در سطح HTTP بررسی می‌کند. عملیات OMDb در محیط تست mock می‌شوند تا نتیجه آزمون به شبکه یا سهمیه API وابسته نباشد.

۳۴ روش جداسازی تست از سرویس خارجی

تست Backend مقدار DATABASE_URL را پیش از import برنامه روی فایل دیتابیس آزمایشی قرار می‌دهد. پیش از هر test، جدول‌ها حذف و دوباره ساخته می‌شوند. lifespan TestClient برنامه را اجرا می‌کند و در نتیجه رفتار ساخت مدیر نیز در محیط تست فعال است.
در تست رسانه، متد خصوصی درخواست OMDb با AsyncMock جایگزین می‌شود. پاسخ اول اطلاعات سریال و دو پاسخ بعدی فصل‌های یک و دو هستند. سپس تست بررسی می‌کند که declaredSeasonCount برابر دو، ترتیب قسمت‌ها درست و تعداد های‌awaitOMDb دقیقاً سه باشد. درخواست دوم همان رسانه نباید await جدیدی ایجاد کند و از این راه رفتار کش تأیید می‌شود.
در SharedPreferences Flutter، با مقادیر mock آغاز می‌شود و FakeBackend حساب و فهرست‌ها را داخل Map نگه می‌دارد. این طراحی هم سرعت تست را بیشتر می‌کند و هم اجازه می‌دهد خطاهای lifecycle بدون اجرای Uvicorn بازتولید شوند. سازگاری JSON میان Dart و Python نیز در اجرای یکپارچه HTTP و بررسی های‌log واقعی Uvicorn ارزیابی می‌شود.

۳۵ ارزیابی دستی ارتباط Client و Server

هنگام اجرای واقعی، های‌log Uvicorn معیار ساده‌ای برای تشخیص مسیر عملیات هستند. جست‌وجو باید درخواست GET روی /api/v1/media/search ایجاد کند. ثبت‌نام و ورود به ترتیب POST روی au-th/register و auth/login هستند. ویرایش پروفایل PATCH روی users/me و ساخت فهرست POST روی me/lists ایجاد می‌کند.
در ارزیابی یکپارچه، ثبت درخواست متناظر برای هر عملیات نشان می‌دهد که مسیر موبایل تا Backend برقرار است. پس از تغییر بخش مقداردهی اولیه احراز هویت، اجرای Flutter به صورت کامل restart می‌شود تا نشست و سرویس‌های جدید از ابتدا ساخته شوند.
برای شبیه‌ساز Android طبیعی است که Flutter آدرس 10.0.2.2 را استفاده کند، در حالی که Backend درخواست ورودی را با آدرس localhost در log نشان دهد. این تفاوت ناشی از ترجمه شبکه شبیه‌ساز است و

به معنی اتصال به دو سرور متفاوت نیست.
دستورهای ارزیابی پروژه عبارت‌اند از:

```
# Flutter
flutter analyze
flutter test

# Backend
cd backend
source .venv/bin/activate
pytest -q
```

امنیت و استقرار

۳۶ ملاحظات امنیتی

رمزها با Argon۲ هش می‌شوند، مسیرهای شخصی به JWT و مسیرهای مدیریتی به نقش مدیر نیاز دارند. کلید OMDb فقط از محیط Backend خوانده می‌شود و پاسخ‌های ورودی با Pydantic اعتبارسنجی می‌شوند. CORS قابل تنظیم است و Android تنها در build اشکال‌زدایی اجازه ترافیک HTTP محلی دارد.
مقادیر پیش‌فرض JWT_SECRET و رمز مدیر صرفاً برای توسعه‌اند. در محیط واقعی باید secrets قوی، HTTPS، دامنه‌های CORS محدود و دیتابیس مناسب مانند PostgreSQL استفاده شوند. توکن نشست موبایل در SharedPreferences نگهداری می‌شود و Header احراز هویت فقط برای درخواست‌های عضو ساخته می‌شود.

۳۷ راه‌اندازی محیط توسعه

Backend یک محیط مجازی Python و وابستگی‌های فایل requirements نیاز دارد. پس از کپی .env.example به .env، کلید OMDb، JWT secret و حساب مدیر تنظیم می‌شوند. اجرای محلی به صورت زیر است:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload
```

پس از اجرا، check health در /health و Swagger در /docs قابل مشاهده‌اند. دیتابیس و جدول‌ها خودکار ساخته می‌شوند. برای Flutter نیز در ریشه پروژه flutter pub get و سپس flutter run اجرا می‌شود. در صورت استفاده از Simulator iOS یا Desktop باید BACKEND_URL روی http://127.0.0.1:8000/api/v1 قرار گیرد.
برای دستگاه فیزیکی، Uvicorn باید با 0.0.0.0 host– اجرا شود و Flutter URL از IP شبکه محلی رایانه استفاده کند. تلفن و رایانه باید در یک شبکه باشند و firewall پورت ۸۰۰۰ را مسدود نکرده باشد. این روش تنها برای توسعه مناسب است و محیط تولید باید دامنه و HTTPS داشته باشد.

۳۸ آمادگی برای محیط تولید

Dockerfile امکان ساخت Backend image را فراهم می‌کند، اما انتشار production فقط با ساخت image کامل نمی‌شود. هاsecret نباید داخل image یا repository قرار گیرند. TLS بهتر است در proxy reverse مانند Nginx یا Caddy خاتمه یابد و ALLOWED_ORIGINS به دامنه‌های واقعی محدود شود.

SQLite برای پروژه درسی و اجرای تک‌نمونه‌ای مناسب است، اما هم‌زمانی زیاد و چند replica به دیتابیس مرکزی مانند PostgreSQL نیاز دارند. مدل‌های SQLAlchemy این تغییر را از طریق DATABASE_URL ممکن می‌کنند. در محیط توسعه، جدول‌ها با create_all ساخته می‌شوند و ساختار SQLAlchemy نقطه مرکزی تعریف schema است.

Uvicorn log، method، path و status و آدرس درخواست را نشان می‌دهد و برای ردیابی ارتباط Flutter و Backend استفاده می‌شود. در تنظیمات انتشار، رمز، JWT و کلید API خارج از log و repository نگهداری می‌شوند.

جمع‌بندی

پروژه فیلم‌یاب یک نمونه چندلایه از اپلیکیشن موبایل است که رابط فارسی Flutter را به Backend اختصاصی FastAPI متصل می‌کند. Backend علاوه بر نقش واسط OMDb، احراز هویت، کنترل نقش، کش رسانه، فعالیت کاربر، فهرست‌های شخصی، گزارش و عملیات مدیریتی را بر عهده دارد. SQLAlchemy امکان استفاده از SQLite در توسعه و تغییر موتور دیتابیس در محیط دیگر را فراهم می‌کند.

در این معماری، Backend مستقل، دیتابیس سرور، JWT، پنهان ماندن کلید خارجی، دریافت واقعی فصل‌ها و قسمت‌ها، پروفایل و فعالیت سرورمحور و نقش مدیر در یک ساختار منسجم کنار یکدیگر قرار گرفته‌اند. نتیجه، سامانه‌ای است که علاوه بر نمایش اطلاعات فیلم و سریال، هویت و فعالیت هر کاربر را نیز به شکل مستقل مدیریت می‌کند.